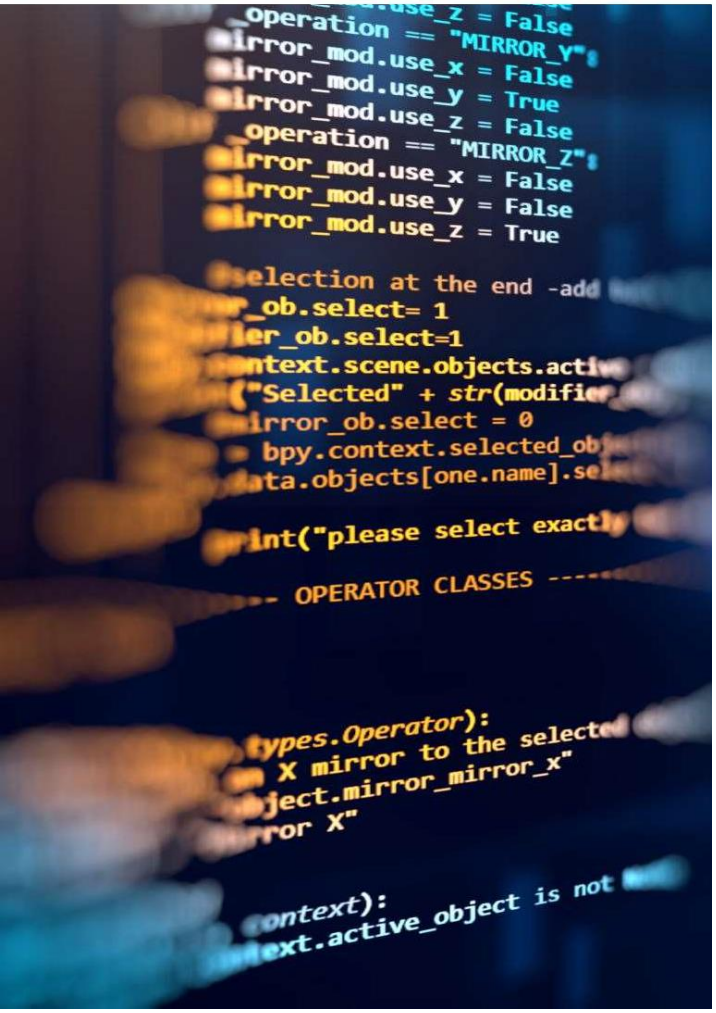
CAMPUS ALEXANDRE BELDI NETTO



Estrutura de Dados

Profº Abimael de Oliveira

Ementa

- **Introdução às estruturas de dados:**
  - Conceitos básicos de estruturas de dados.
  - Importância e aplicações em computação.
- **Listas:**
  - Listas lineares: simplesmente encadeadas, duplamente encadeadas e circulares.
  - Operações básicas: inserção, remoção, busca e ordenação.
- **Pilhas e Filas:**
  - Definição e características.
  - Implementações estáticas e dinâmicas.
  - Aplicações e algoritmos associados.
- **Árvores:**
  - Árvores binárias: características e propriedades.
  - Percursos em árvores: pré-ordem, em ordem, pós-ordem.
- **Grafos:**
  - Definição e representação.
  - Busca em largura e busca em profundidade.
- **Análise de Algoritmos:**
  - Noções de complexidade temporal e espacial.
  - Análise de algoritmos de ordenação e busca.
- **Aplicações Práticas:**
  - Implementação de algoritmos e estruturas de dados em linguagens de programação.
  - Resolução de problemas utilizando estruturas de dados.
  - Avaliação de desempenho e otimização de algoritmos.

Avaliações

$$Média\ Final = (AC1 * 0,15) + (AC2 * 0,30) + (AG * 0,10) + (AF * 0,45)$$

Avaliação	Descrição	Peso média final
AC1	Avaliação Continuada 1 Média da nota das atividades desenvolvidas conforme Plano de Aulas ao longo do período	15%
AC2	Avaliação Continuada 2 Média da nota das atividades desenvolvidas conforme Plano de Aulas ao longo do período	30%
AG	AG – Avaliação Geral, De acordo com as orientações institucionais	10%
AF	Avaliação Final Prevista no calendário institucional avaliação individual envolvendo todo o conteúdo da componente curricular.	45%

Bibliografia Básica

1. PEREIRA, Silvio do Lago. Estruturas de Dados em C - Uma Abordagem Didática. Erica, 2016. ISBN Digital: 9788536517254.
2. CORMEN, T. H.; LEISERDON, C. E.; RIVEST, R. L.; STEIN, C. Algoritmos. 3ª ed. Rio de Janeiro: Editora Campus, 2012.
3. SOFFNER, Renato. Algoritmos e Programação em Linguagem C [recurso eletrônico, Minha Biblioteca]. 1ª edição. São Paulo: Saraiva, 2013.

Bibliografia Complementar

1. MIZRAHI, Victorine Viviane. Treinamento em linguagem C. 2. ed. São Paulo, SP: Pearson Prentice Hall, 2008. 405 p. ISBN 9788576051916.
2. CASTRO, J., Linguagem C na prática. São Paulo, Ciência Moderna, 2008.
3. SOFFNER, Renato. Algoritmos e programação em linguagem C. São Paulo Saraiva 2013 1 recurso online ISBN 9788502207530.
4. TENENBAUM, Aaron M.; LANGSAM, Yediyah; AUGENSTEIN, Moshe J. Estruturas de dados usando C. São Paulo, SP: Makron Books, 1995. 884 p. ISBN 8534603480..
5. ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene Aparecida Veneruchi de. Fundamentos da programação de computadores: algoritmos, Pascal e C/C++. São Paulo, SP: Prentice-Hall, 2003. 355 p. ISBN 8587918362.

Estrutura de Dados

- **Estrutura de Dados**, é um modo particular de **armazenamento e organização de dados** em um computador, de modo que possam ser **usados eficientemente**, facilitando sua busca e modificação.

- Utilizados nas mais diversas áreas do conhecimento e com os mais diferentes propósitos de aplicação.

- As técnicas e estruturas que permitem **armazenar os dados de forma ordenada e organizada**, aumentam o **desempenho dos algoritmos**, além de **minimizar** o uso de **memória e processamento**.

Unidade básica de armazenamento

Bits	Bytes	Valores Decimais
8 bits	1 byte	-128 a 127
16 bits	2 bytes	- 32.768 a 32.767
32 bits	4 bytes	-2.147.483.648 a 2.147.483.647
64 bits	8 bytes	18.446.744.073.709.551.616

Tipos Primitivos em Java

		Valores possíveis				
Tipos	***Primitivo***	***Menor***	***Maior***	***Valor Padrão***	***Tamanho***	***Exemplo***
Inteiro	byte	-128	127	0	8 bits	byte ex1 = (byte)1;
	short	-32768	32767	0	16 bits	short ex2 = (short)1;
	int	-2.147.483.648	2.147.483.647	0	32 bits	int ex3 = 1;
	long	-9.223.372.036.854.770.000	9.223.372.036.854.770.000	0	64 bits	long ex4 = 1l;
Ponto Flutuante	float	-1,4024E-37	3.40282347E + 38	0	32 bits	float ex5 = 5.50f;

	double	-4,94E-307	1.79769313486231570E + 308	0	64 bits	double ex6 = 10.20d;
						ou
						double ex6 = 10.20;

Caractere	char	0	65535	\0	16 bits	char ex7 = 194; ou char ex8 = 'a';
Booleano	boolean	false	true	false	1 bit	boolean ex9 = true;

Linguagem JAVA

- O **Java** como linguagem de programação **nasceu em 1991** pelas mãos de um pesquisador da **Sun Microsystems**, James Gosling.

- A **ideia original** do projeto era desenvolver uma **linguagem de programação** que fosse **independente de plataforma** e que pudesse ser executada em diferentes dispositivos eletrônicos, como **geladeiras**, **televisões** ou **fogões**.

- Inicialmente, o projeto não foi prioridade para a Sun, já que alguns executivos acharam **absurda a ideia** de ter uma linguagem que pudesse ser **processada numa televisão.** Com a chegada da TV digital interativa, podemos ver que não estavam tão errados assim...

- Sendo uma **linguagem** de **programação orientada a objetos**, robusta, elegante, podendo ser utilizada nos mais diversos ambientes.

Máquina Virtual Java (JVM)

- A forma de execução da linguagem Java é baseada na interpretação por meio de uma máquina virtual, a **Java Virtual Machine (JVM).** Ela proporciona um ambiente de aplicação completo para execução de aplicativos Java.

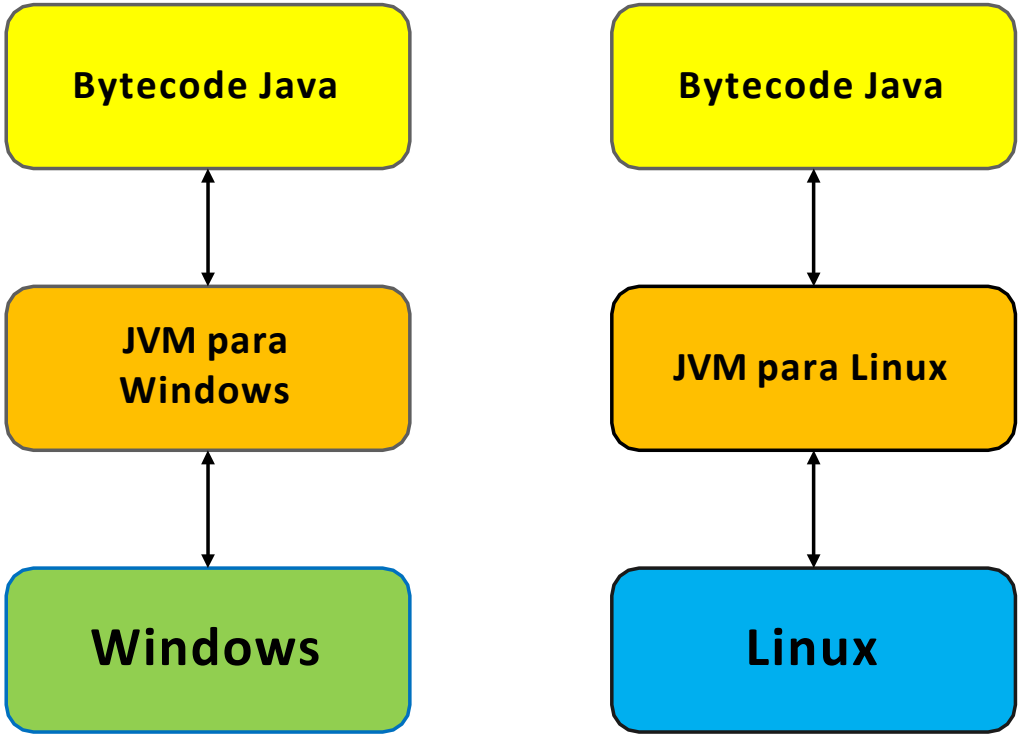
- **Java Virtual Machine (JVM)**
  - *Imagine uma máquina criada por meio de um software emulador dentro de uma máquina real, como se fosse um minicomputador dentro do seu próprio computador, e que possui gerenciamento de memória, cache, threads e outros processosde um computador com seu sistema operacional.*

- A compilação de um código fonte Java é o ***bytecode***, que é uma linguagem de máquina inteligível para a **JVM**.

- O **bytecode** é independente de hardware; assim, basta o computador ou o dispositivo eletrônico ter o interpretador adequado (a **JVM**) que poderá executar um programa Java.

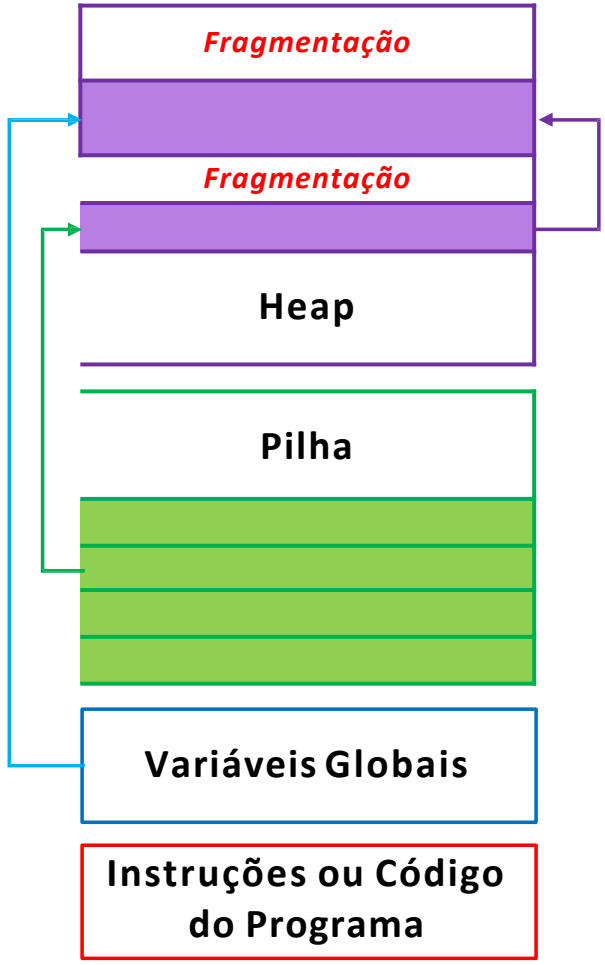
Máquina Virtual Java (JVM)

- *O conceito de **máquina virtual** garante a **independência do código** em relação ao sistema operacional.*
- *E aumenta a segurança da aplicação, principalmente em servidores.*
- *O **Bytecode é gerado** por um compilador Java, como o **javac**.*
- *A **JVM compila dinamicamente** durante a execução do código, podendo **aplicar estratégias de otimização.***



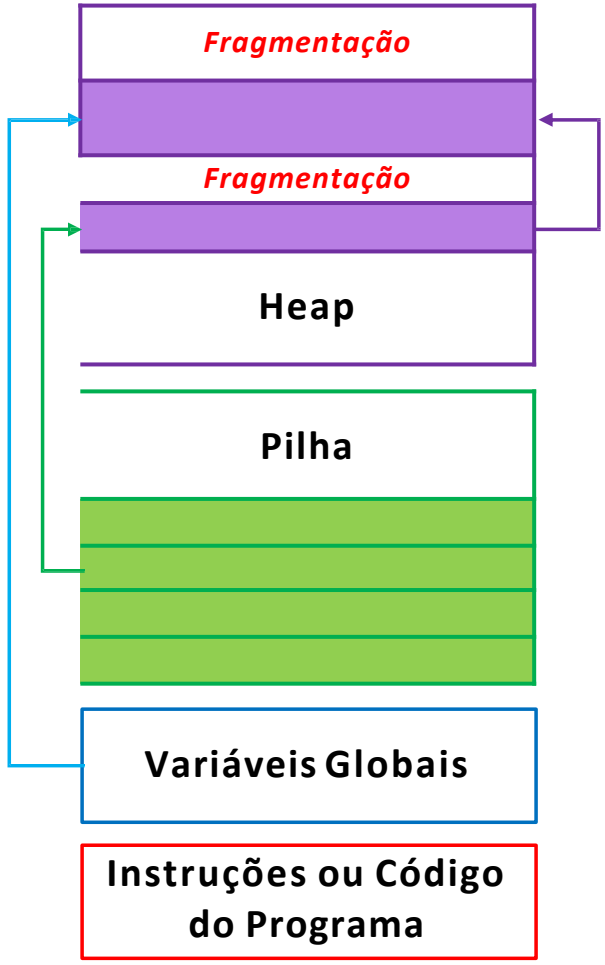
Alocação de Memória

- A maioria das linguagens de programação utilizam um esquema de segmentação de memória em quatro regiões logicamente distintas.
  - **Instruções:** utilizado para armazenar o código do programa sendo executado.
  - **Variáveis Globais:** utilizado para armazenar variáveis declaradas no escopo inicial do programa, como por exemplo constantes.
  - **Pilha:** utilizado para armazenar as variáveis locais e ponteiros de endereçamento para variáveis dinâmicas
  - **Heap:** utilizado para armazenar variáveis alocadas dinamicamente.



Alocação de Memória

- **Escopo de duração das variáveis:**
  - **Variáveis Globais:** espaço de memória continua alocada enquanto o programa estiver executando.
  - **Variáveis locais:** espaço de memória continua alocada enquanto a função que a declarou estiver executando.
- **Espaço para alocação:**
  - **Variáveis Estáticas:** o tamanho do espaço em memória para a alocação da variável é conhecida.
  - **Variáveis Dinâmicas:** o tamanho do espaço em memória para a alocação é desconhecido, compilador cria um espaço no **Heap** e registra o endereço em um ponteiro na **Pilha.**

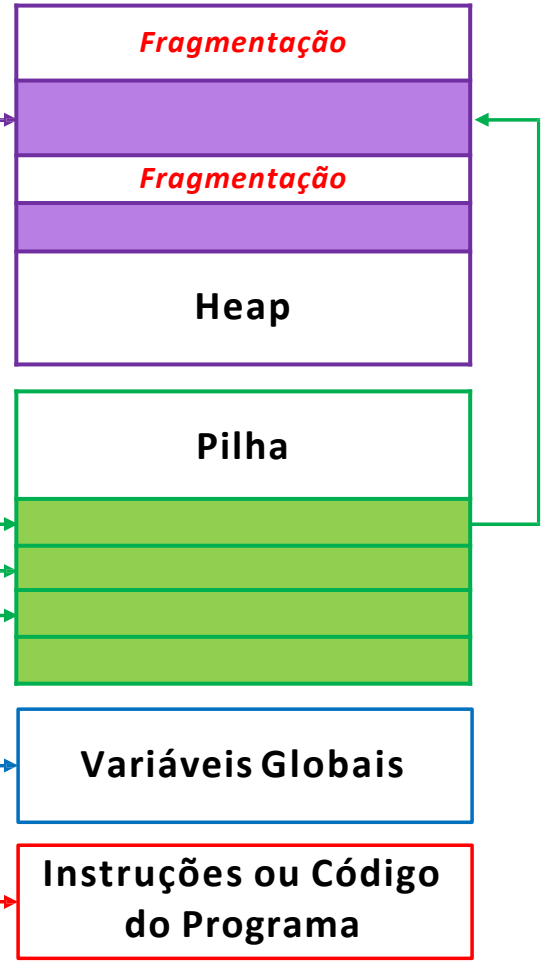


Alocação de Memória

```
public static void main(String[] args) {

        double pi = 3.1415;
        double raio = 2.00
        double area;
        StringBuffer nome = new StringBuffer();

        area = calculaArea(raio);
}
public static double calculaArea(double r){
        double a;
        a = 2* pi * sqrt(r);
        return a;
        }
```



19/02/2024

Alocação de Memória

Processo de limpeza:

- Em linguagens de programação com C/C++ o processo de limpeza da memória alocada é responsabilidade do programador.
- Na **linguagem C** existe 4 funções especificas para isso:
  - **Malloc**: permite a alocação de uma nova área de memória para uma variável, retornando um ponteiro para o endereço de memória.
  - **Calloc**: similar ao Malloc, mas a variável é inicializada com zero.
  - **Realoc**: permite realocar o tamanho da área ocupada pela variável.
  - **Free**: permite que uma área alocada seja liberada.

Alocação de Memória

Processo de limpeza:

- Em Java e Python o programador não precisa gerenciar explicitamente a memória do sistema.
- Pois existem mecanismos automáticos de gerenciamento e limpeza da memória, conhecido como coletor de lixo (**Garbage Collector**).
- A técnica de Coleta de Lixo consiste na recuperação segura do espaço de memória ocupado por um objeto que não é mais referenciado dentro de uma aplicação.
- Em linguagens orientadas a objeto é importante que objetos que não são mais referenciados, sejam desalocados para liberar a memória ocupada.

Alocação de Memória

Processo de limpeza:

- **Exemplo:**

StringBuffer nome= new StringBuffer();

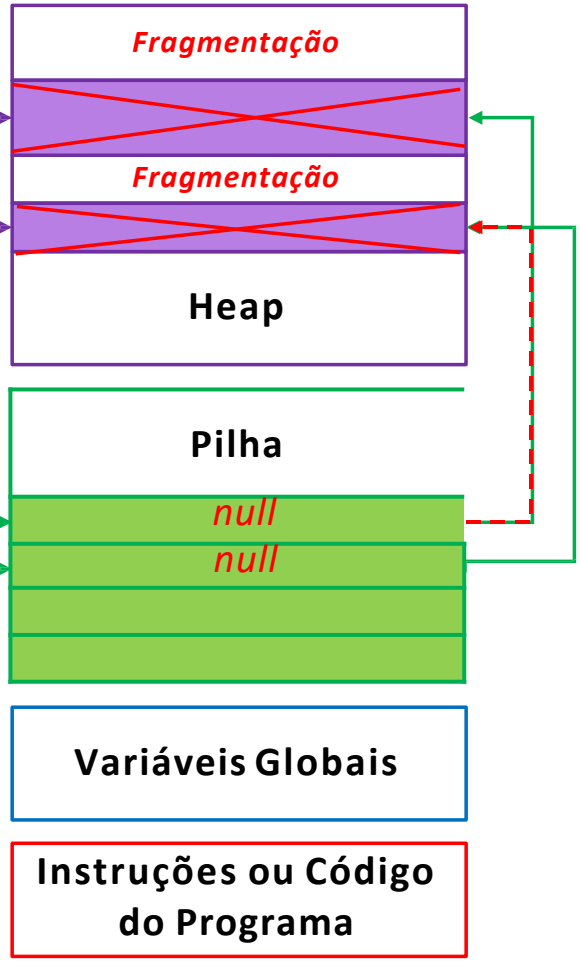
StringBuffer auxiliar= new StringBuffer();

⋮

nome= auxiliar;

⋮

nome= auxiliar = null;



Alocação de Memória

Garbage Collector:

- **Possui a função:**
  - Alocar a memória.
  - Assegurar que quaisquer objetos referenciados permaneçam na memória.
  - Recuperar a memória alocada pelos objetos que não são mais alcançáveis pelas referências do código em execução.

- Em geral, quando não há interferência do programador, a máquina virtual executa o Coletor de Lixo quando percebe que a memória está ficando sem espaço.

Alocação de Memória

Garbage Collector:

- De qualquer maneira, uma aplicação pode encerrar sua execução sem que o **Garbage Collector** seja executado uma única vez.
- Quando um objeto deixa de ser referenciado, o espaço de memória ocupado por ele **não é imediatamente desalocado**.
- O **Garbage Collector** pode ser configurado para priorizar diferentes comportamentos:
  - Diminuir a quantidade de intervenções utilizando assim mais memória.
  - Realizar limpezas mais constantes otimizando o uso de memória.

Alocação de Memória

```
public static void main(String[] args) {
	Runtime rt = Runtime.getRuntime();
	System.out.println("Memoria alocada para a JVM: " + rt.totalMemory()+" Byte");
	rt.gc();
	System.out.println("Memoria alocada para a JVM apos executar o GC: " + rt.totalMemory()+" Byte");
	double m1,m2;
	m1= rt.freeMemory();
	System.out.println("Memoria disponivel para criacao dos objetos: " + rt.freeMemory()+" Byte");

	double vetor[]= new double[100000];

	m2= rt.freeMemory();
	System.out.println("Memoria disponivel depois da criacao dos objetos: " + rt.freeMemory()+" Byte");
	System.out.println("Memoria utilizada: " + (m1- m2) + " bytes");
	rt.gc(); //Executa o Garbage Collector
	System.out.println("Memoria depois de executar o Garbage Collector: " + rt.freeMemory()+" Byte");
}
```

Vetores

- **Considerem o seguinte problema:** *ler uma certa quantidade de valores inteiros e os imprimir na ordem inversa da leitura.*
    - *Entrada (2, 5, 3, 4, 9) □ Saída (9, 4, 3, 5, 2)*

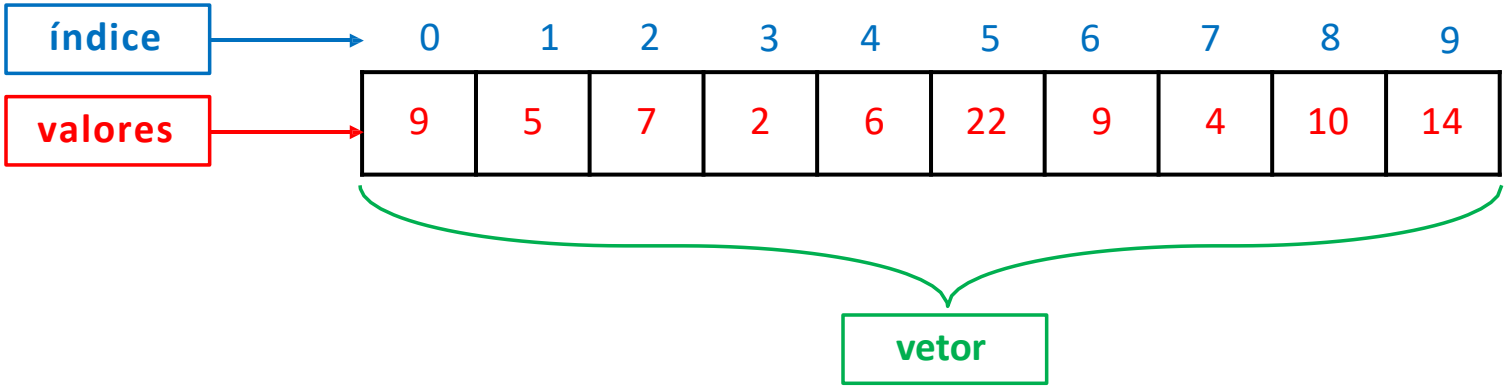
- Este tipo de problema é impossível de ser resolvido com o uso de uma única variável pois, quando se lê o segundo número, já se perdeu o primeiro da memória. Exigindo o uso de tantas variáveis quantos fossem os dados de entrada, mas notem que isto deve ser conhecido em *tempo de compilação*!

- Quando se aloca, um número inteiro em uma variável, o que ocorre é que o computador reserva uma posição de memória em algum endereço da memória RAM. Com um número inteiro exigindo 4 bytes no caso do Java.

Vetores

- Os **vetores** são estruturas de dados que permitem o acesso a uma grande quantidade de dados em memória utilizando somente um nome de variável.

- Para acessar o conteúdo de uma posição especifica do vetor é necessário a informação do índice do vetor.



Vetores

Declaração e atribuição em Java

int[] vetor = new[]{9,5,2.....,14};

int[] vetor = new[10];

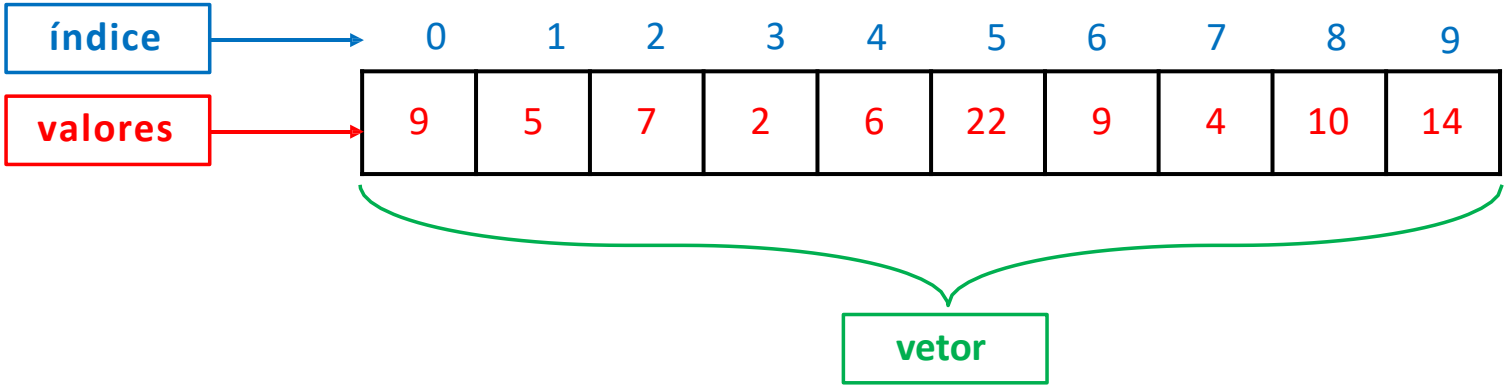
vetor[0] = 9;

vetor[1] =5;

.....

Acessando os dados:

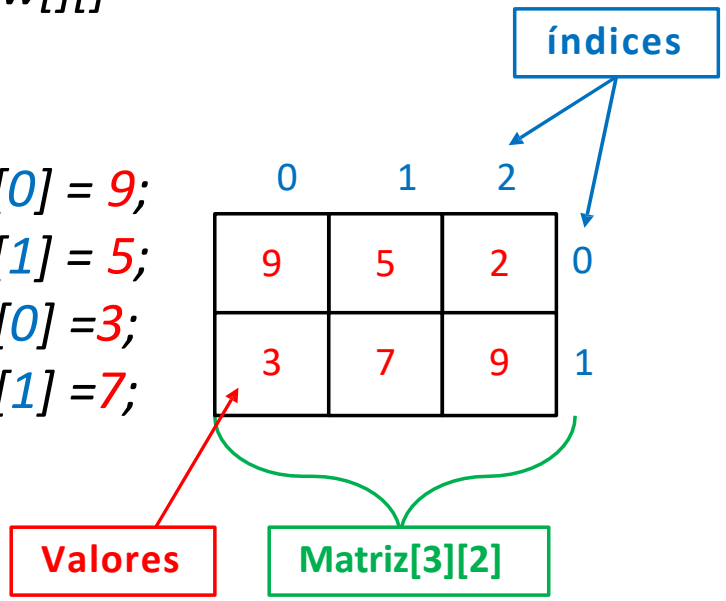
= vetor[0];



Matrizes

Declaração e *int[][] matriz = new[][]*
{{9,5,2},{3,7,9}};
atribuição em Java *int[][] matriz = new[3][2];*

matriz[0][0] = 9;
matriz[0][1] = 5;
matriz[1][0] =3;
matriz[1][1] =7;

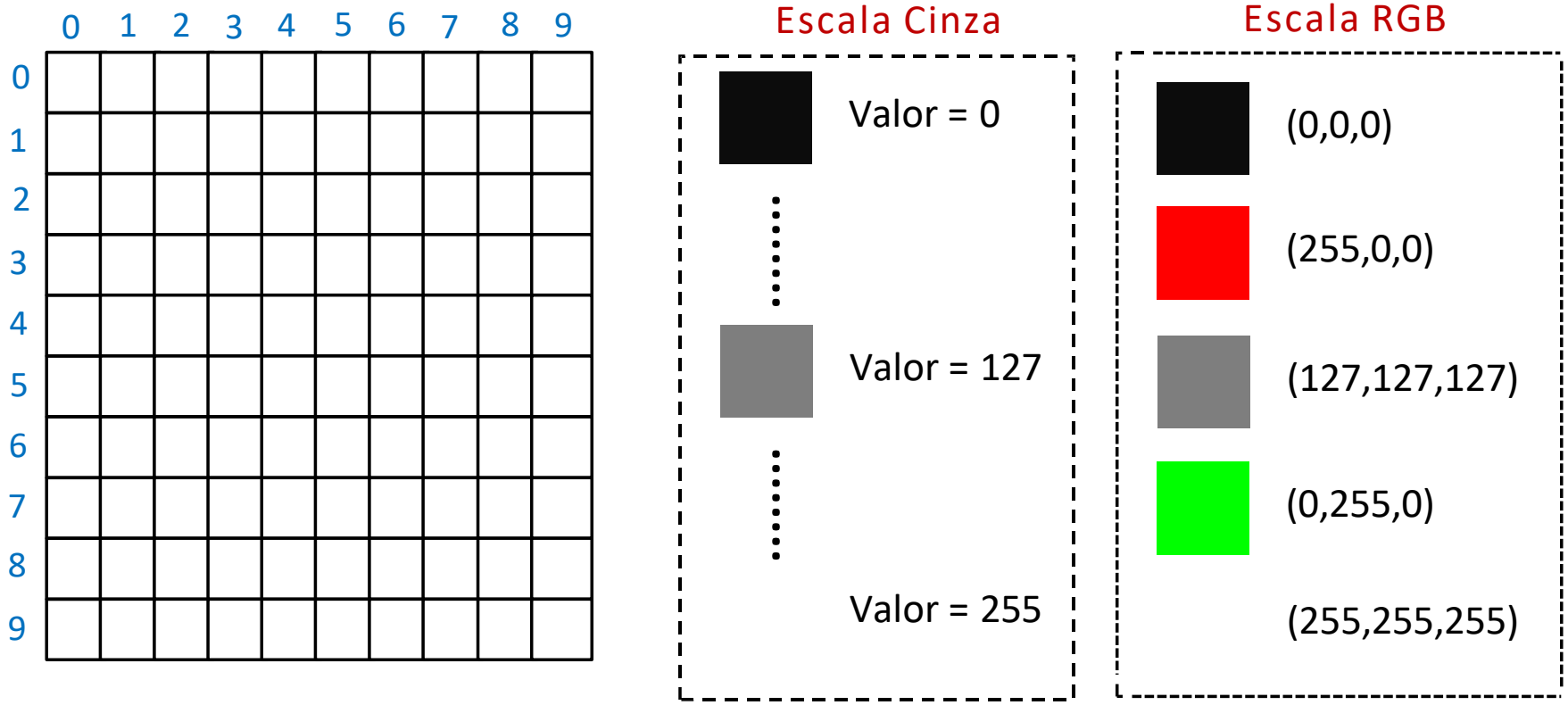


Acessando os dados: *= matriz[0][0];*

Na linguagem Java o tamanho dos vetores e matrizes é fixo, não podendo ser alterado em *tempo de execução*. Linguagens como C e VB, permitem o redimensionamento durante a execução do código.

Vetores e Matrizes

Onde posso utilizar uma matriz de valores ???



Vetores e Matrizes

Exercício

1. Crie um método que receba 1 vetor com 8 posições, inverta a ordem dos itens e retorne o vetor invertido.

2. Crie um método que receba 3 vetores com 8 posições cada e retorne uma matriz de 8x3 posições com os respectivos valores armazenados nos vetores.

3. Crie um método que receba como parâmetro um número binário na forma de um vetor com 8 posições e retorne esse número convertido para decimal.

4. Crie um método que receba como parâmetro um número decimal, converta esse número para binário e retorne na forma de um vetor com 8 posições.

* Para obter o resto de uma divisão utilize o operador %. *Exemplo: int resto = 9 % 2*

Exercício

5. Escreva um método que receba dois vetores de 5 números cada, gere um terceiro vetor com os elementos dos outros dois vetores intercalados.
   Exemplo: vetor 1º ={A,B,C,D,E} |2º={1,2,3,4,5} |3º={A,1,B,2,C,3,D,4,E,5}

5. Escreva um método para imprimir no console uma matriz de 2 dimensões. Utilize o seguinte padrão.

X
Y
Z

X	X
Y	Y
Z	Z

Utilize System.out.print("|" + "\t" + variável + "\t" + "|") para imprimir no console os marcadores com o espaço TAB entre eles.

Exercício

7. Crie um método que dado uma matriz [3x2] gere sua equivalente [2x3] transposta. Conforme exemplo, utilize o algoritmo do exercício 6 para imprimir a matriz original e a transposta.

$$B_{3x2} = \begin{bmatrix} 1 & 5 \\ 7 & 3 \\ 8 & 2 \end{bmatrix} \qquad B^t_{2x3} = \begin{bmatrix} 1 & 7 & 8 \\ 5 & 3 & 2 \end{bmatrix}$$

Sistema Decimal e Binário

Decimal

10^3	10^2	10^1	10^0
1	5	2	1

$$1 * 10^3 + 5 * 10^2 + 2 * 10^1 + 1 * 10^0 =$$

$$1000 + 500 + 20 + 1 = \mathbf{1521}$$

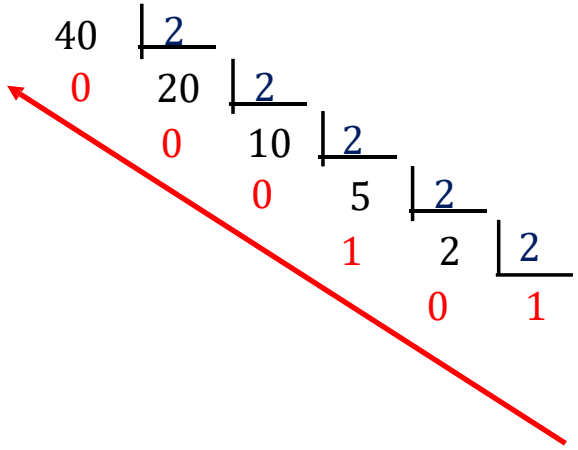
Binário

2^3	2^2	2^1	2^0
1	1	0	1

$$1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 =$$

$$8 + 4 + 0 + 1 = \mathbf{13}$$

Conversão Decimal para Binário.



1 0 1 0 0 0 (*binário*)

Conversão Binário para Decimal.

2^5 2^4 2^3 2^2 2^1 2^0

32+ 0 + 8 + 0 + 0 + 0 = 40